Lund University, Faculty of Engineering
Neo4j

Graph Neural Networks on Large Graphs via Graph Databases

A thesis submitted in partial satisfaction
of the requirements for the degree
Master of Science in Computer Science

by

Victor Pekkari, Harry Wik

2026

Abstract of the Thesis

Graph Neural Networks on Large Graphs via Graph Databases

by

Victor Pekkari, Harry Wik
Master of Science in Computer Science
Lund University, Faculty of Engineering
Neo4j, 2026

Graph-structured data appears in domains such as fraud detection, recommendation, and molecular analysis, which has made graph neural networks (GNNs) a central approach for learning on graphs. However, message passing creates dependencies between nodes that make large-scale training and inference difficult, especially when the graph does not fit comfortably in memory. Existing large-scale GNN systems commonly rely on distributed training and graph partitioning, but these approaches can suffer from communication overhead and expensive sampling.

This thesis investigates whether a graph database can serve as an efficient and scalable backend for GNN training and inference. More specifically, it studies a Neo4j-backed pipeline integrated with PyTorch Geometric and evaluates the bottlenecks of database-backed training, the efficiency of different sampling strategies, the best approach for inference on new nodes, and how the usefulness of the approach changes with graph size. The work follows an experimental, systems-oriented methodology where components of a functionally equivalent PyTorch Geometric pipeline are replaced, benchmarked, and analyzed.

The thesis contributes a structured system design for graph-database-backed GNN training, a methodology for evaluating both correctness and throughput, and an analysis framework for understanding when graph databases are competitive with more conventional large-scale GNN systems.

The thesis of Victor Pekkari, Harry Wik is approved.

Lund University, Faculty of Engineering

Neo4j, 2026

Table of Contents

1 Introduction . . . **1**

1.1 Research questions . . . 2

1.2 Scope and limitations . . . 2

1.3 Thesis outline . . . 2

2 Background . . . **3**

2.1 Graphs and graph representation learning . . . 3

2.1.1 Transductive nature of traditional GRL . . . 3

2.2 Neural networks . . . 4

2.3 Graph neural networks . . . 4

2.3.1 Message passing . . . 5

2.3.2 Implications for large-graph training . . . 5

2.4 Graph neural networks for large graphs . . . 6

2.4.1 Training under memory constraints . . . 6

2.4.2 Subgraph sampling . . . 7

2.5 PyTorch Geometric . . . 8

2.6 Neo4j and graph databases . . . 8

2.6.1 Alternatives to Neo4j . . . 9

2.6.2 Querying Neo4j with Cypher and procedures . . . 9

2.6.3 Data retrieval from Neo4j . . . 9

2.6.4 Cypher query execution pipeline . . . 9

3 Related Work . . . **11**

3.1 Subgraph sampling . . . 11

3.2 Distributed GNN systems . . . 11

3.3 Database-backed GNN pipelines . . . 12

3.4 Research gap . . . 12

4 Method . . . **13**

4.1 Experimental methodology . . . 13

4.2 Baseline graph-database-backed GNN system . . . 13

4.3 System configurations . . . 13

4.4 Datasets . . . 14

4.5 Equivalence constraint . . . 14
4.6 Performance evaluation . . . 14
4.7 Benchmarking procedure . . . 15
4.8 Graph neural network model . . . 15
4.9 Sampling algorithms . . . 15
4.9.1 Neighbor sampling . . . 15
4.9.2 GraphSAINT random-walk sampling . . . 16

5 System Design . . . 17
5.1 Baseline architecture . . . 17
5.2 Improved architecture . . . 18
5.3 Graph representation in Neo4j . . . 18
5.4 PyTorch Geometric integration . . . 18
5.5 Profiling and logging . . . 19
5.5.1 Training logging . . . 19
5.5.2 Query execution profiling . . . 19
5.5.3 Python-level profiling . . . 19
5.6 Implementation decisions . . . 19

6 Results . . . 20
6.1 RQ1: Bottlenecks . . . 21
6.1.1 Training phase vs. sampling phase . . . 21
6.1.2 Sampling subphases . . . 21
6.1.3 Feature vector datatype . . . 21
6.1.4 Effect of caching . . . 21
6.1.5 Effect of multisampler parallelism . . . 21
6.1.6 Summary of RQ1 . . . 21
6.2 RQ2: Sampling strategies . . . 21
6.2.1 Neighbor sampling in Neo4j . . . 21
6.2.2 GraphSAINT in Neo4j . . . 21
6.2.3 Summary of RQ2 . . . 21
6.3 RQ3: Inference with graph stored in Neo4j . . . 21
6.3.1 Inference setting definitions . . . 21
6.3.2 Batch inference performance . . . 21
6.3.3 Inference on newly inserted nodes . . . 21

6.3.4 Summary of RQ3 21
6.4 RQ4: Effect of graph size 21
6.4.1 Small graphs 21
6.4.2 Medium graphs 21
6.4.3 Large graphs 21
6.4.4 Summary of RQ4 21

7 Discussion 22
7.1 Optimal system implementation 22
7.2 Implications 22
7.3 Threats to validity 22

8 Conclusion 23
8.1 Future work 23

Acknowledgments

We would like to thank the Graph Data Science team at Neo4j for hosting this thesis work and for providing access to infrastructure, expertise, and an invaluable working environment. We are also grateful to our supervisor and examiner at LTH for their guidance throughout the project.

CHAPTER 1

Introduction

Graph structures are a natural way to model entities and their relationships in several important domains. Such models are applied in diverse domains, including predicting chemical bonds in medical science and detecting fraud in finance. The prevalence of graph-structured data across many domains creates a need for efficient and accurate methods to model and analyze graphs.

Graph neural networks (GNNs) have emerged as a leading approach for representation learning on graphs, combining the structural intuition behind classical graph algorithms with the expressiveness and inductive capabilities of neural networks.

However, GNNs introduce a challenge absent in traditional neural networks: message passing creates growing dependencies between nodes during training and inference.

Existing approaches for scaling GNNs to large graphs rely on partitioning the graph, sampling subgraphs, and distributing computation across machines. These approaches often suffer from high communication overhead between machines, and from inefficient sampling procedures that can take longer than the actual model training.

In practice, many large graphs are already stored in graph databases before they are loaded into workers for GNN training or inference. The characteristics of fast graph traversal and practically unlimited memory in graph databases make them a promising component in a GNN training and inference pipeline for large graphs. While it has been shown that it is possible to use a graph database as a backend for GNN training, prior work has mainly demonstrated feasibility rather than performance competitiveness with state-of-the-art distributed systems. It is therefore still unclear (i) what the bottlenecks in graph-database-backed training and inference are, (ii) what an efficient graph-database-based pipeline would look like, (iii) whether and when graph-database-backed training pipelines might be a better choice than distributed systems, and (iv) how inference should be done with the graph stored in a graph database.

This thesis investigates whether a graph database can serve as an efficient and scalable backend for GNN training and inference on large graphs. To answer this, we implement and evaluate a Neo4j-backed pipeline integrated with PyTorch Geometric, analyze its main bottlenecks, compare sampling strategies, and study how the usefulness of this approach depends on graph size and inference setting.

1.1 Research questions

This thesis addresses the following research questions:

1. **RQ1 — Bottlenecks:** What are the primary bottlenecks in GNN training when using a graph database for graph storage, and to what extent can they be mitigated?
2. **RQ2 — Sampling:** Which subgraph sampling strategies are most efficient when the graph is stored in a graph database?
3. **RQ3 — Inference:** What is the optimal method for doing inference on new nodes when the graph is stored in a graph database?
4. **RQ4 — Graph size:** For which graph sizes does a graph database-based storage approach provide the greatest benefits?

1.2 Scope and limitations

The experiments in this thesis are conducted on four datasets of varying sizes using a two-layer Graph Convolutional Network (GCN) for node classification. The main emphasis is on the training and inference pipeline rather than on model architecture. The study is therefore limited to a specific GNN setting, but targets system-level questions that are relevant to large-scale graph learning more broadly.

1.3 Thesis outline

Chapter 2 provides background on important topics needed for understanding the rest of the thesis, such as GNNs, neural networks, graph representation learning, Neo4j, and PyTorch Geometric. Chapter 3 reviews related work on GNNs applied to large graphs. Chapter 4 describes the methodology used during the experiments. Chapter 5 outlines the implemented GNN pipelines. Chapter 6 describes the results, Chapter 7 discusses them, and Chapter 8 concludes the thesis.

CHAPTER 2

Background

Graphs in mathematics are used to model pairwise relationships between objects. Objects are modeled as nodes $\mathcal{V}$, and their relations are called edges $\mathcal{E}$. Many domains benefit from modeling data as objects (nodes) and relationships (edges). Consider, for example, social networks, protein–drug interactions, or railway networks. Studying graph structure and node features allows us to predict properties of the graph.

When making predictions about graphs it is often useful to start by creating an embedding of the graph. The optimal embedding depends on the downstream task. There are several ways to embed a graph, transductively with traditional graph representation learning (GRL) approaches, and inductively with GNNs.

The purpose of this chapter is to give a fundamental understanding of traditional GRL algorithms, neural networks, and how GNNs combine ideas from traditional graph representation learning and neural networks. From traditional GRL, GNNs inherit the idea that node embeddings should reflect graph structure, while from neural networks they adopt a learnable, parameterized function. Challenges of working with GNNs on large datasets that do not exist in traditional neural networks are also discussed.

Lastly, a basic introduction to `PyTorch Geometric`, the machine learning library used during experiments, and `Neo4j`, a graph database that provides useful graph operations for training GNNs, is provided.

2.1 Graphs and graph representation learning

In graph representation learning the aim is to find a representation of the graph that encodes its structure. Embeddings like this are often used in downstream tasks such as node classification, link prediction, or clustering. Finding a good embedding depends in part on what the downstream task is.

2.1.1 Transductive nature of traditional GRL

A common feature throughout traditional graph representation learning algorithms, whether based on node statistics, matrix factorization, or random walks, is that no weights are shared between nodes and no node-embedding generating function is learned. Instead, only node embeddings for the current nodes in the graph are learned. This means that if new nodes are introduced to the graph, there is no direct way to compute their embeddings, and the

embeddings for the old nodes are no longer accurate. The latter also happens if nodes disappear from the graph.

Consider a simplification of the optimization problem for random-walk-based algorithms:

$$\min_{\{z_x \,:\, x \in \mathcal{V}\}} \sum_{(u,v)\in\mathcal{D}} -\log\left(\sigma(\mathbf{z}_u^\top \mathbf{z}_v)\right)$$

where $\mathcal{D}$ is a set of node pairs encountered on the same random walk, and $\mathcal{V}$ is the set of nodes in the graph.

The optimization problem above is an example of how it is the actual node embeddings that are optimized, rather than some embedding-generating function. This is what makes traditional representation learning algorithms transductive in nature.

Consider a social network where people are nodes, and their relations are edges. If a transductive graph representation is used, the learned feature vectors are tied to specific people in the graph. If more people are introduced to the network, the feature vector for every person has to be recomputed.

2.2 Neural networks

Neural networks are parameterized functions that can learn a function mapping inputs to outputs. Given an input x, a neural network produces an output $\hat{y}$ according to a function $f_\theta(x)$, where θ are the parameters, also known as weights, of the model.

Optimizing the weights θ of the model includes minimizing a loss function $\mathcal{L}$ which measures how far the model prediction is from the desired target values. This optimization is usually done using some gradient-based method.

A key property of neural networks is that they are inductive: instead of learning representations tied to specific data points, they learn a general function that can be applied to unseen inputs. This enables generalization beyond the training data.

Standard neural network training further relies on the assumption that data points are independent. Under this assumption, the loss can be approximated using mini-batches of data, allowing efficient and scalable training. However, this assumption does not hold for graph-structured data, where data points (nodes) are explicitly dependent on each other through the graph topology. This limitation motivates the development of graph neural networks, which extend neural networks to account for relational dependencies.

2.3 Graph neural networks

Graph neural networks (GNNs) are neural networks designed for graph-structured data. Like traditional graph representation learning methods, they aim to produce node representations that reflect graph structure. Unlike shallow graph embedding methods, however, GNNs learn

a shared parameterized function that generates node representations from node features and graph connectivity. This makes GNNs inductive: once trained, the same model can be applied to unseen nodes, and in some cases to entirely new graphs drawn from a similar distribution.

A central idea in GNNs is *message passing*. Instead of computing a node representation only from the node's own features, a GNN updates each node by aggregating information from its neighbors. By stacking several such layers, the model can incorporate information from increasingly distant parts of the graph.

2.3.1 Message passing

At message passing layer k, the representation of node v is updated as

$$\mathbf{x}_v^{(k)} = \phi^{(k)}\left(\mathbf{x}_v^{(k-1)}, \bigoplus_{u \in \mathcal{N}(v)} \psi^{(k)}\left(\mathbf{x}_v^{(k-1)}, \mathbf{x}_u^{(k-1)}\right)\right),$$

where $\mathcal{N}(v)$ denotes the neighbors of node v, $\psi^{(k)}$ is a message function, $\bigoplus$ is a permutation-invariant aggregation operator such as sum or mean, and $\phi^{(k)}$ is an update function.

After K layers, the representation of a node depends on information from its K-hop neighborhood. This allows GNNs to exploit both node features and graph structure when solving downstream tasks such as node classification.

2.3.2 Implications for large-graph training

The message-passing mechanism also creates the main computational challenge in GNN training. In standard neural networks, data points in a mini-batch are usually treated as independent, so the loss and gradient can be computed from the batch alone. In a GNN, this is no longer true: the representation of a node depends on its neighbors, whose representations in turn depend on their neighbors. As a result, computing the output for a batch of seed nodes generally requires access to a larger induced subgraph around those nodes.

This dependency grows with the number of GNN layers. For a K-layer model, each seed node may depend on a K-hop neighborhood, which can become very large in real-world graphs. Training on the full graph is therefore often infeasible for memory and computation reasons, especially for large graphs. To address this, practical GNN pipelines typically rely on subgraph or neighborhood sampling, where only a bounded part of the neighborhood is retrieved for each mini-batch.

2.4 Graph neural networks for large graphs

In typical neural networks, data points are either independent or have dependencies with a clear and structured form. This structure is determined by the architectural inductive bias, which makes it straightforward to derive which data points influence each other. For example, in recurrent neural networks, t_n depends on $\{t_{n-1}, t_{n-2}, \dots\}$, while in convolutional neural networks a pixel $p_{x,y}$ depends only on a local neighborhood $\{p_{i,j} \mid |i-x| \leq r,\ |j-y| \leq r\}$ in the same image. In both cases, the set of dependent data points is explicitly defined and bounded.

In contrast, graph neural networks lack such explicit structural assumptions. The dependency structure is determined by the graph itself, making it harder to determine which data points influence a given node. Moreover, these dependencies are not only less structured, but also significantly larger. The representation of a node depends on its k-hop neighborhood, which can grow exponentially with k. As a result, each data point in a GNN may depend on orders of magnitude more data points than in models such as convolutional neural networks, where dependencies are limited to small local regions.

2.4.1 Training under memory constraints

In standard neural networks, the independence, or structured bounded dependence, between data points allows training on datasets larger than the available memory. Data can be processed in virtual mini-batches $\{d^{(1)}, \dots, d^{(n)}\}$ such that

$$\sum_i d_x^{(i)} < Y,$$

where Y is the available memory. Crucially, the gradient computed on a batch depends only on the data points within that batch. This assumption breaks for GNNs as the output for one node is dependent on an unbounded number of neighbors. The gradient for the seed nodes in the batch cannot be computed accurately without their neighbors. To solve this, the neighbors of the seed nodes are fetched at the beginning of every batch to build a subgraph around the seed nodes so that the gradient for that batch can be computed accurately.

Algorithm 1: Mini-batch GNN Training (Single Epoch)

```
// Mini-batch training with neighbor-sampled subgraphs
Input: f_θ, V
Output: θ
for each batch B ⊆ V do
    G_B ← ∅
    for each seed node v ∈ B do
        (X_v, y_v, E_v) ← SAMPLE(v)
        G_B ← G_B ∪ (X_v, y_v, E_v)
    ŷ ← f_θ(X_B, E_B)
    L ← CROSSENTROPY(ŷ, y_B)
    θ ← θ − η∇_θ L
return θ
```

2.4.2 Subgraph sampling

To address this, GNN training typically relies on subgraph sampling. Instead of using full neighborhoods, a bounded subset of neighbors is sampled for each node. Given a batch of seed nodes, a subgraph is constructed by sampling up to a fixed number of neighbors at each hop. This limits the size of the computation graph and enables mini-batch training.

A common approach is layer-wise neighbor sampling, where upper bounds are defined as $[k_0, k_1, \ldots, k_{K-1}]$. At layer i, at most k_i neighbors are sampled per node. If the number of neighbors exceeds this bound, a subset is selected, typically uniformly at random.

While sampling makes training tractable, it introduces an approximation of the true neighborhood aggregation. Consider the exact aggregation for a node v, using the mean of node-neighbor feature vectors as the aggregation function:

$$h_v = \sigma\left(\frac{1}{|\mathcal{N}(v)|}\sum_{u\in\mathcal{N}(v)} x_u\right),$$

where σ is a nonlinear activation function.

With sampling, we instead compute:

$$\tilde{h}_v = \sigma\left(\frac{1}{k}\sum_{u\in\tilde{\mathcal{N}}(v)} x_u\right),$$

where $\tilde{\mathcal{N}}(v)$ is a sampled subset of neighbors.

Although the sampled mean is an unbiased estimator of the true mean,

$$\mathbb{E}\left[\frac{1}{k}\sum_{u\in\tilde{\mathcal{N}}(v)} x_u\right] = \frac{1}{|\mathcal{N}(v)|}\sum_{u\in\mathcal{N}(v)} x_u,$$

the presence of the nonlinearity introduces bias:

$$\mathbb{E}[\tilde{h}_v] = \mathbb{E}[\sigma(\cdot)] \neq \sigma(\mathbb{E}[\cdot]) = h_v.$$

This discrepancy arises because σ is nonlinear, and therefore expectation does not commute with the activation function. In practice, this bias is often small and does not prevent convergence, especially for shallow GNNs.

2.5 PyTorch Geometric

PyTorch Geometric is a graph machine learning library built on top of PyTorch. It provides implementations of various message-passing layers, as well as abstractions for storing the graph.

- `FeatureStore:` defines how the node features are stored and retrieved.
- `GraphStore:` defines how the graph is traversed, and how edges are stored.
- `BaseSampler:` defines how neighborhood sampling is done, and is tightly coupled with the graph store.
- `NodeLoader:` takes implementations of `FeatureStore`, `GraphStore`, and `BaseSampler` and provides a data loader.

2.6 Neo4j and graph databases

Databases are systems used to efficiently store and fetch data. Instead of scanning raw files, databases provide a way to find relevant data with low latency. Traditional databases are relational, and the data is stored in the form of tables of rows and columns. Relationships between objects are stored through foreign keys, which leads to expensive join operations when retrieving relational data. This model is not well suited for queries that involve repeated traversal of relationships, which is something needed for fast subgraph sampling in GNN training.

Graph databases such as Neo4j are, in contrast, optimized to have efficient operations on graphs. This is done by explicitly modeling nodes and edges, making relationships first-class citizens. This allows queries that follow connections, such as multi-hop traversals, to be expressed and executed more naturally and efficiently. In an ETL pipeline to a GNN model it is necessary that the database supports efficient retrieval, and scalability in the form of horizontal scaling via clustering as well as vertical scale-up of the machines running the database instances.

2.6.1 Alternatives to Neo4j

Although Neo4j is a market leader, capturing the largest market share in the graph database space, there are several alternatives. Amazon Neptune, Azure Cosmos, and Google Spanner Graph are examples that mainly compete with Neo4j by tight integration with the respective cloud platforms. Other examples include Memgraph and TigerGraph.

2.6.2 Querying Neo4j with Cypher and procedures

Cypher is a declarative query language for graph databases where queries are expressed as pattern matches over nodes and edges. The query planner translates the Cypher code into execution plans optimized for traversal and lookup. In this work, Cypher is used to express neighborhood expansion queries required for subgraph sampling.

Procedures extend Cypher with imperative logic that cannot be expressed in Cypher. Procedures are defined in Java within the database engine, allowing custom traversal strategies, early termination, and more fine-grained control over execution. This makes procedures a relevant alternative when trying to optimize highly specialized Cypher queries. Procedures are explored alongside Cypher when fetching neighbors of the seed nodes.

2.6.3 Data retrieval from Neo4j

Data is retrieved in batches determined by the `fetch_size`. A smaller `fetch_size` results in more round trips between the client and the database. For each batch, a number of rows corresponding to the `fetch_size` are first located in the database and then serialized before being transmitted over the network. The serialization speed depends on the underlying data format; for example, byte arrays are typically faster to serialize than more complex structures. While one batch is being transmitted, the next batch is prepared in the database, serialized, and subsequently sent to the client.

2.6.4 Cypher query execution pipeline

The incoming Cypher string is parsed into an abstract syntax tree. The syntax tree is then validated, for example by checking for labels, relationship types, or property predicates that do not exist. The next step is normalization into a canonical form.

The normalized syntax tree is used to generate one or more logical plans: operator trees that define how to produce the query response. Much like the expression

$$R := c + a \cdot b$$

is reduced according to operator precedence, a logical plan generated by a Cypher query acts upon the data source with operators such as `NodeByLabelScan` and `Filter`. One logical plan can often be resolved according to many different physical plans, all yielding the same output given the input. A simpler query is likely to only have one physical plan whereas a more

complex query can have many. In the case of multiple physical plans, they are compared against each other so that a cost metric is minimized. Examples of elements weighed into the cost optimization are metadata such as how many nodes have a certain label and whether an index lookup should be used.

After cost optimization one physical plan is promoted to be the execution plan. The execution step compiles optimized bytecode that can be used and reused. To answer the query in a timely manner, the execution step can have a parallel runtime.

Some features are restricted to paying database clients, that is, clients that run Neo4j Enterprise rather than Neo4j Community. Parallelism is one such example along with clustering, extra constraints, online backups, and role based access control.

Regardless of parallelism, execution first attempts to retrieve from the page cache in RAM. The nodes and relations required to resolve the query are either found in cache through pointer chasing, or a page fault occurs and data needs to be loaded from non-volatile memory, for example an NVMe RAID disk. The exact method of retrieval depends on the storage format. The current default is block format.

After data has been localized, it needs to be sent back to the client. First, the binary representations are cast to proper types. Next is the removal of non-return variables. Before finally returning data to the client, the Java classes must be serialized. Neo4j has a Bolt server that utilizes PackStream to binary encode the data, after which the standard TCP/IP stack takes over and answers the query.

CHAPTER 3

Related Work

Existing approaches to scaling GNN training can be broadly categorized as sampling-based methods and distributed systems. Sampling methods reduce computational cost by approximating neighborhoods, while distributed systems improve scalability through parallelism at the cost of communication overhead.

3.1 Subgraph sampling

Sampling methods are widely used when training GNNs because of the exploding neighborhoods that come from stacked message-passing layers. Sampling-based methods sample subgraphs from the larger original graph. This reduces both the computational and memory cost.

GraphSAGE introduced layer-wise neighbor sampling, where a fixed number of neighbors are sampled per node during training. **GraphSAINT** instead samples subgraphs using random walks or node-based sampling strategies, aiming to improve training efficiency and reduce variance.

While these methods make large-scale GNN training tractable, they introduce approximation error due to incomplete neighborhood information, and their performance depends on the efficiency of the sampling procedure.

3.2 Distributed GNN systems

Another line of work focuses on distributed GNN training, where the graph is partitioned across multiple machines. These systems aim to scale training by parallelizing both computation and data access.

[insert about papers here]

Despite their scalability, distributed approaches often suffer from communication overhead between machines and require careful partitioning strategies to balance load and minimize data transfer.

3.3 Database-backed GNN pipelines

A smaller body of work explores graph-database-backed GNN training. Here the memory storage and fast graph traversal in graph databases is leveraged, instead of storing the graph in memory across several machines.

Lopushanskyy and Shi demonstrate that it is possible to train GNNs directly on a graph database by retrieving only the required subgraphs during training.

However, existing work in this area has primarily focused on demonstrating feasibility, with limited evaluation of performance compared to state-of-the- art distributed systems. In particular, it remains unclear what the main bottlenecks are and under what conditions database-backed approaches are competitive.

3.4 Research gap

The literature shows that both sampling-based and distributed approaches can scale GNN training, but it leaves open several questions about graph-database- backed pipelines. In particular, prior work does not sufficiently explain what their dominant bottlenecks are, which sampling strategies are most effective in this setting, how inference should be performed when the graph remains in the database, or for which graph sizes such an approach is most beneficial. These questions define the research gap addressed in this thesis.

CHAPTER 4

Method

This research study was conducted at Neo4j, within the Graph Data Science team. It follows an experimental systems-oriented methodology, where different GNN training and inference pipelines are tested in an ablational manner. The primary objective in the experiments is to find a GNN pipeline that keeps functional equivalence to PyTorch Geometric while maximizing throughput.

4.1 Experimental methodology

The study is based around a graph-database-backed GNN pipeline that has been proven to have functional equivalence to PyTorch Geometric. Components of the GNN pipeline are modified and benchmarked to identify performance bottlenecks and potential optimizations.

Each experiment isolates a single design choice, such as sampling strategy, data retrieval method, or caching mechanism, allowing its impact on performance to be evaluated independently. This enables a systematic exploration of the design space of graph-database-backed GNN training.

4.2 Baseline graph-database-backed GNN system

As a reference, a PyTorch Geometric-based training pipeline is used. This pipeline defines the expected behavior of sampling, batching, and model execution, and serves as a ground truth for correctness. The Neo4j-backed implementation replaces the data loading and sampling components of this pipeline while keeping the model architecture and training loop identical.

4.3 System configurations

Experiments are conducted across multiple system configurations, where both hardware setup and pipeline design may vary. These configurations include:

- Different sampling implementations (Cypher queries vs. procedures)
- Variations in data retrieval strategies (batch size, serialization format)
- Use of caching mechanisms in feature retrieval

- Parallel vs. single-sampler execution

Each configuration represents a distinct point in the design space of graph-database-backed GNN training.

4.4 Datasets

The datasets, consisting of homogeneous graphs, used for training and evaluating GNNs for node classification were:

- **ogbn-arxiv**, 170k nodes: a directed graph representing the citation network between computer science arXiv papers indexed by MAG.
- **ogbn-products**, 2.4M nodes: an undirected and unweighted graph representing an Amazon product co-purchasing network.
- **ogbn-papers100M**, 111M nodes: a directed citation graph of 111 million papers indexed by MAG.

4.5 Equivalence constraint

A key requirement throughout all experiments is that the Neo4j-backed pipeline remains functionally equivalent to the PyTorch Geometric reference implementation. Equivalence is evaluated using:

- Consistency in sampled subgraph structure statistics, such as the number of sampled edges and nodes at each K-hop
- Similar training convergence behavior
- Comparable validation and test accuracy
- Distributional similarity of sampled neighborhoods at each K-hop

Only configurations that satisfy these criteria are considered valid, ensuring that performance improvements do not come at the cost of altered model behavior.

4.6 Performance evaluation

The primary performance metric is training throughput, measured as the number of processed seed nodes per unit time. This is complemented by a detailed breakdown of pipeline stages, including sampling time and feature retrieval time. In addition, database-level metrics such as query latency, number of database hits, and serialization overhead are collected to identify system bottlenecks.

4.7 Benchmarking procedure

A dedicated measurement framework is used to collect timing and profiling data during training. Each experiment is run for multiple epochs to ensure stable measurements. Comparisons between configurations are made under identical conditions, including a fixed model architecture, consistent hyperparameters, and identical datasets. This ensures that observed performance differences are attributable solely to system design choices.

4.8 Graph neural network model

All experiments use a two-layer Graph Convolutional Network (GCN). The model applies two `GCNConv` message-passing layers with ReLU activations and a final linear classifier. The architecture is intentionally kept simple to focus evaluation on the data pipeline rather than the model itself. The model parameters that vary are the dimension of the first `GCNConv` layer, which matches the dimension of the node feature vectors, and the output dimension, which matches the number of classes in the dataset.

4.9 Sampling algorithms

The GraphSAGE sampler and the GraphSAINT sampler were implemented both in Cypher and in Java using user-defined procedures in Neo4j. The sampling implementations were then evaluated using the methodology described in this chapter, maximizing throughput while keeping functional equivalence to the true sampling algorithm. The code for all sampling implementations can be found in the appendix.

4.9.1 Neighbor sampling

For each seed node in the mini-batch, the sampler issues a Cypher query that traverses the graph up to K hops, sampling at most $[k_0, k_1, \ldots, k_{K-1}]$ neighbors at each hop. Neighbors exceeding the limit are selected uniformly at random without replacement. This corresponds to the GraphSAGE sampling strategy.

Algorithm 2: Neighbor Sampling

```
Input: Seed nodes S, neighbor limits [k_0, k_1, ..., k_{K-1}]
Output: Sampled node set V_sub
V_sub ← S
begin ← 0
end ← |S|
for ℓ ← 0 to K − 1 do
    k ← k_ℓ
    for i ← begin to end − 1 do
        v ← V_sub[i]
        Sample up to k neighbors of v uniformly at random, with or without replacement
        Add sampled neighbors to V_sub if not already present
    begin ← end
    end ← |V_sub|
return V_sub
```

4.9.2 GraphSAINT random-walk sampling

Instead of expanding neighborhoods around seed nodes, GraphSAINT constructs a subgraph by first sampling a set of nodes via random walks and then inducing the subgraph over those nodes.

CHAPTER 5

System Design

The system consists of three main components: a Neo4j graph database acting as the graph and feature store, a Python training process that implements the PyTorch Geometric abstract interfaces, and a `NodeLoader` that orchestrates mini-batch construction during training. The implementations of `FeatureStore` and `GraphStore` act as the interface between the training process and the Neo4j database. The implementation of `BaseSampler` defines how the subgraph sampling is done.

5.1 Baseline architecture

The baseline GNN pipeline has three main components: (i) Neo4j to store the graph, traverse the graph, and retrieve node feature vectors, node labels, and data split information; and (ii) a Python process that implements the required interfaces for abstract graph storage and also the model forward pass and weight updates.

The Neo4j database is used to store both the graph structure and the features of the nodes. The Python training process can therefore, instead of materializing the full graph in memory, retrieve the data needed for only the current mini-batch. For each batch of seed nodes, the Python process issues queries to Neo4j through the implemented PyTorch Geometric interfaces.

Within the Python process, the `Neo4jGS` implementation is responsible for graph traversal and neighborhood expansion, while `Neo4jAbstractFS` handles feature retrieval. The `Neo4jSampler` coordinates these components to produce sampled subgraphs that are functionally equivalent to those expected by the standard PyTorch Geometric training pipeline. The resulting architecture preserves the usual mini-batch GNN training loop, but replaces in-memory graph access with on-demand retrieval from the database.

The baseline system is deployed on a single Google Cloud Compute Engine virtual machine (`n2-highmem-32`) with 32 vCPUs and 256 GB RAM. Both the Neo4j database and the Python training process run on the same machine. This design avoids inter-machine communication and makes it possible to isolate the overhead introduced by database-backed sampling and feature retrieval.

5.2 Improved architecture

The improved architecture builds on the baseline design by optimizing the most important parts of the data path: neighborhood sampling, serialization, feature retrieval, and parallel batch preparation. The concrete improvements are introduced and evaluated incrementally in the experiments rather than assumed up front.

5.3 Graph representation in Neo4j

Nodes in Neo4j are stored with a label corresponding to the node type, for example `Paper` for citation networks. Each node carries the following properties:

- A unique integer id.
- Pre-computed node feature vectors. The vectors can either be stored as `Float64[]` or `bytes[]`.
- A target class label for supervised node classification, represented as an integer.
- A split indicator, such as `train`, `val`, or `test`, used to partition the dataset.

Edges are stored as typed relationships connecting pairs of nodes. No edge features are used in the experiments.

5.4 PyTorch Geometric integration

Three PyTorch Geometric abstract classes described in Section 2.5 were implemented.

`Neo4jAbstractFS` is an abstract class extending `FeatureStore`. The purpose of this class is to provide storage of the feature vectors, for example in a graph database, and to implement an efficient interface for fetching feature vectors. The reason it is kept abstract is that while there are some methods any Neo4j-interacting PyTorch Geometric `FeatureStore` needs, such as executing queries, there are also some features that may vary depending on the use case. One example of this is caching. Avoiding Neo4j fetches by storing commonly used features in RAM is a good optimization, but how this caching is done in the feature store should be up to the user. There are also two extensions of `Neo4jAbstractFS`: `Neo4jCachedFS` and `Neo4jNoCacheFS`.

`Neo4jAbstractGS` is an abstract class extending `GraphStore`, with the main goal of implementing efficient traversal over the graph to enable fast subgraph sampling. It has two implementations, `Neo4jMultiGS` and `Neo4jSingleGS`, where the difference is how many samplers they are optimized to handle.

`Neo4jSampler` implements the sampling algorithm, and is tightly coupled with the graph store where it executes the sampling algorithm.

5.5 Profiling and logging

5.5.1 Training logging

Measurements are continuously taken during training and stored in a structured format for later analysis.

5.5.2 Query execution profiling

Cypher queries are instrumented with the `PROFILE` keyword to collect operator-level statistics: rows produced, database hits, estimated memory usage, and query execution time. These are aggregated across batches and stored as averaged CSV output.

5.5.3 Python-level profiling

Call-stack profiles are collected using Python's `cProfile` module and stored as `train_profile.txt`. These are used to identify hot spots in the Python data pipeline that are not visible in the query-level metrics, such as tensor allocation overhead or unnecessary data copies.

5.6 Implementation decisions

Important design tradeoffs in the system concern where to spend memory, where to accept approximation, and how tightly to couple the PyTorch Geometric interfaces to Neo4j-specific capabilities. Caching improves throughput but uses more RAM. Procedures can outperform declarative queries for specialized sampling, but increase implementation complexity and reduce portability. Similarly, multi-sampler execution can improve concurrency, but it may also increase contention in the database and complicate equivalence guarantees.

CHAPTER 6

Results

This chapter presents the experimental results from graph-database-backed GNN training and inference. The evaluation focuses on which phases dominate runtime during training or inference, as well as total throughput and functional correctness in the sampling.

6.1 RQ1: Bottlenecks

6.1.1 Training phase vs. sampling phase

6.1.2 Sampling subphases

6.1.3 Feature vector datatype

6.1.4 Effect of caching

6.1.5 Effect of multisampler parallelism

6.1.6 Summary of RQ1

6.2 RQ2: Sampling strategies

6.2.1 Neighbor sampling in Neo4j

6.2.2 GraphSAINT in Neo4j

6.2.3 Summary of RQ2

6.3 RQ3: Inference with graph stored in Neo4j

6.3.1 Inference setting definitions

6.3.2 Batch inference performance

6.3.3 Inference on newly inserted nodes

6.3.4 Summary of RQ3

6.4 RQ4: Effect of graph size

6.4.1 Small graphs

6.4.2 Medium graphs

6.4.3 Large graphs

6.4.4 Summary of RQ4

CHAPTER 7

Discussion

This chapter discusses the meaning of the results in relation to the research questions and the broader goal of assessing graph databases as a backend for large-scale GNN training and inference.

7.1 Optimal system implementation

The discussion centers on which combination of sampling strategy, feature retrieval method, caching, and execution model provides the best tradeoff between throughput, simplicity, and correctness.

7.2 Implications

The implications are both practical and methodological. Practically, the work helps clarify when database-backed GNN pipelines are a viable alternative to in-memory or distributed systems. Methodologically, it contributes a way of evaluating such systems under a strict equivalence constraint to a PyTorch Geometric reference implementation.

7.3 Threats to validity

Threats to validity include the use of a single model family, a limited set of datasets, hardware-specific effects, and implementation choices tied to Neo4j and PyTorch Geometric. These constraints limit the generality of the results, but they are acceptable given that the thesis focuses on a concrete systems problem rather than universal model performance.

CHAPTER 8

Conclusion

This thesis studies whether a graph database can act as an efficient and scalable backend for GNN training and inference. By implementing a Neo4j-backed pipeline integrated with PyTorch Geometric, the work investigates bottlenecks, sampling efficiency, inference strategies, and how the usefulness of the approach changes with graph size. The thesis contributes both a concrete system design and an evaluation methodology for comparing graph-database-backed GNN pipelines against a functionally equivalent reference implementation.

8.1 Future work

Future work includes broadening the evaluation to more GNN architectures, testing additional graph databases, extending the system to more distributed deployments, and studying how database-backed inference behaves under stricter latency requirements and dynamic graph updates.